

Scipy sparse matrix alternative for getrow()

Asked 6 years, 5 months ago Modified 2 years, 2 months ago Viewed 2k times



5



I am working with large sparse binary matrices. I have condensed them using `scipy` sparse matrix implementation. The calculation of `Jaccard` distance from `scipy.spatial.distance` does not support direct operation on sparse matrices, so either:

1. convert the entire sparse matrix to dense and then operate on each row as vectors which is **memory hungry**
or
2. Loop through the sparse, grab each row using `getrow()` and operate.
or
3. Write our own implementation to work on sparse matrices.

To put things to perspective, here is the sample code:

```
import scipy.spatial.distance as d
import numpy as np
from scipy.sparse import csr_matrix

# benchmark performance
X = np.random.random((3000, 3000))
# binarize
X[X > 0.3] = 0
X[X>0] = 1
mat = csr_matrix(X)

a = np.zeros(3000)
a[4] = a[100] = a[22] = 1
a = csr_matrix(a)

def jaccard_fast(v1,v2):
    common = v1.dot(v2.T)
    dis = (v1 != v2).getnnz()
    if common[0,0]:
        return 1.0-float(common[0,0])/float(common[0,0]+dis)
    else:
        return 0.0

def benchmark_jaccard_fast():
    for i in range(mat.shape[0]):
        jaccard_fast(mat.getrow(i),a)

def benchmark_jaccard_internal_todense():
    for v1,v2 in zip(mat.todense(),a.todense()):
        d.jaccard(v1,v2)

def benchmark_jaccard_internal_getrow():
    for i in range(mat.shape[0]):
        d.jaccard(mat.getrow(i),a)

print "Jaccard Fast:"
%time benchmark_jaccard_fast()
print "Jaccard Scipy (expanding to dense):"
```

```
%time benchmark_jaccard_internal_todense()
print "Jaccard Scipy (using getrow):"
%time benchmark_jaccard_internal_getrow()
```

where `jaccard_fast` is my own implementation. It appears that my implementation is faster than the internal one, on scipy sparse matrices, however `getrow()` seems to slow my implementation down. As I benchmark `jaccard_fast` against `scipy.spatial.distance.jaccard`, results are:

```
Jaccard Fast:
CPU times: user 1.28 s, sys: 0 ns, total: 1.28 s
Wall time: 1.28 s
Jaccard Scipy (expanding to dense):
CPU times: user 28 ms, sys: 8 ms, total: 36 ms
Wall time: 37.2 ms
Jaccard Scipy (using getrow):
CPU times: user 1.82 s, sys: 0 ns, total: 1.82 s
Wall time: 1.81 s
```

Any help on how to avoid the `getrow` bottleneck would be appreciated. I cannot afford to expand my sparse matrix using `todense()` due to memory limitations.

[python](#) [scipy](#) [sparse-matrix](#) [Edit tags](#)

Share Edit Follow Close Flag

Protect

edited Jun 20, 2020 at 9:12

asked May 6, 2016 at 11:16



Community Bot
1 1



Segmented
1,874 2 21 39

▲ I think your `benchmark_jaccard_internal_todense` is cheating. It's not doing the full 3000 iterations. `a.todense()` is (1,3000), so zip on it and `mat` iterates on the least number of rows - ie. 1. - [hpaulj](#) May 6, 2016 at 15:51

▲ `benchmark_jaccard_internal_getrow` isn't a valid test since by your own admission `d.jaccard` does not work with sparse. - [hpaulj](#) May 6, 2016 at 18:24

2 Answers

Sorted by:
[Reset to default](#)

Date modified (newest first) ◆

Try this:

-1

```
def getrow_using_underlying_repr(a):
    for i,j in enumerate(a.indptr[:-1]):
        a.data[a.indptr[i]:a.indptr[i+1]]
```



It's more than 100 times faster than `getrow()`:

```
Jaccard Fast:
CPU times: user 1.41 s, sys: 0 ns, total: 1.41 s
Wall time: 1.41 s
Jaccard Scipy (expanding to dense):
CPU times: user 19.9 ms, sys: 29.8 ms, total: 49.6 ms
Wall time: 49.3 ms
Jaccard Scipy (using getrow):
CPU times: user 1.48 s, sys: 1.27 ms, total: 1.48 s
Wall time: 1.48 s
getrow_using_underlying_repr:
CPU times: user 11 µs, sys: 1e+03 ns, total: 12 µs
Wall time: 13.8 µs
```

To understand the code see the explanation in the last version of the instantiation in the documentation:

https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csr_matrix.html#scipy.sparse.csr_matrix

To understand sparse CSR's underlying representation better see:

<https://www.geeksforgeeks.org/sparse-matrix-representations-set-3-csr/amp/?ref=rp>

By investigating the the implementation of built-in getrow() in the source, it turns out that the reason of the slowness is that it doesn't use the underlying representation, but the matrix product of CSR sparse matrices.

Share Edit Follow Delete Flag

edited Aug 10, 2020 at 15:59

answered Aug 10, 2020 at 14:17



4



```
In [33]: timeit for row in mat: x=row # sparse iteration
1 loops, best of 3: 510 ms per loop
```



```
In [35]: timeit for row in mat.todense(): x=row # dense iteration
10 loops, best of 3: 175 ms per loop
```



but I find that your `d.jaccard` is also slower when using sparse matrices

```
In [36]: ad=a.todense()
```

```
In [37]: timeit for row in mat.todense(): d.jaccard(row,ad) # all dense
1 loops, best of 3: 734 ms per loop
```

```
In [38]: timeit for row in mat: d.jaccard(row.todense(),ad) # inner dense
1 loops, best of 3: 1.69 s per loop
```

```
In [39]: timeit for row in mat: d.jaccard(row,a) # all sparse
1 loops, best of 3: 4.61 s per loop
```

Eliminating the `getrow` factor

```

In [40]: mrow=mat.getrow(0)

In [41]: mrowd=mrow.todense()

In [42]: timeit d.jaccard(mrow, a) # one sparse row
1000 loops, best of 3: 1.32 ms per loop

In [43]: timeit d.jaccard(mrow.todense(), a.todense()) # with conversion
1000 loops, best of 3: 539 µs per loop

In [44]: timeit d.jaccard(mrowd, ad) # dense
10000 loops, best of 3: 173 µs per loop

```

=====

I need to rerun those tests because `d.jaccard` does not work with sparse (and `jaccard_fast` does not work with dense). So separating the sparse row iteration issues from the `jaccard` calculation will take more work.

I've reworked `jaccard_fast` a bit:

```

def my_jaccard(mat, a):
    common = mat*a.T # sparse does the large matrix product well
    dis=np.array([(row!=a).getnnz() for row in mat]) # iterative
    cA = common.A.ravel()
    return 1 - cA/(cA + dis)

```

It returns values that match `d.jaccard` run on the dense rows. `d.jaccard` returns 1 for the rows where `common` is 0. I don't seem to need a `cA` test (unless it is possible that both `cA` and `dis` are 0 at the same slot).

```

In [141]: r=np.array([d.jaccard(row,ad) for row in mat.todense()])

In [142]: r1=my_jaccard(mat,a)

In [143]: np.allclose(r,r1)
Out[143]: True

```

and speed is only half. If I can rework the `dis` calc it should have similar speed.

```

In [144]: timeit r=np.array([d.jaccard(row,ad) for row in mat.todense()])
1 loops, best of 3: 783 ms per loop

In [145]: timeit r1=my_jaccard(mat,a)
1 loops, best of 3: 1.29 s per loop

```

A further tweak to the calc. I mask out the `common` values that are 0. This has 2 purposes - it ensures we don't have a divide by 0 problem, and it reduces the number of `dis` iterations, giving a small speed improvement.

```

def my_jaccard(mat, a):
    common=mat*a.T
    cA = common.A.ravel()

```

```

mask = cA!=0
cA = cA[mask]
dis = np.array([(row!=a).getnnz() for row, b in zip(mat,mask) if b])
ret = np.ones(mat.shape[0])
ret[mask] = 1 - cA/(cA+dis)
return ret

```

with this the time drops a bit.

```

In [188]: timeit my_jaccard(mat,a)
1 loops, best of 3: 1.04 s per loop

```

=====

There's an overlap in issues with [Python - Efficient Function with scipy sparse Matrices](#)

In that question I looked at comparing a sparse matrix with a 1 row matrix, and found that using `sparse.kron` to replicate the row, was the fastest way to replicated `numpy` broadcasting.

Using that idea in `jaccard` to calculate the `dis` array

```

def my_jaccard1(mat, a):
    common = mat*a.T
    cA = common.A.ravel()
    aM = sparse.kron(a,np.ones((mat.shape[0],1),int))
    dis = (mat!=aM).sum(1)
    ret = 1-cA/(cA+dis.A1)
    return ret

```

With this timings improve significantly (10x):

```

In [318]: timeit my_jaccard1(mat,a)
1 loops, best of 3: 97.1 ms per loop

```

I can apply masking as before to protect against divide by zero; but it actually slows down the calculation (to 140ms).

```

def my_jaccard3(mat, a):
    common = mat*a.T
    cA = common.A.ravel()
    mask = cA!=0
    cA = cA[mask]
    aM = sparse.kron(a,np.ones((len(cA),1),int))
    dis = (mat[mask,:]!=aM).sum(1)
    ret = np.ones(mat.shape[0])
    ret[mask] = 1 - cA/(cA+dis.A1)
    return ret

```

=====

edit - test of the suspected case

```
In [75]: x,y= np.array([1,1,0,0,1,0]), np.array([0,0,1,0,1,0])

In [76]: d.jaccard(x,y)
Out[76]: 0.75

In [78]: jaccard_fast(sparse.csr_matrix(x),sparse.csr_matrix(y))
Out[78]: 0.75
```

My versions:

```
In [79]: my_jaccard(sparse.csr_matrix(x),sparse.csr_matrix(y))
Out[79]: array([ 0.75])
...
In [82]: my_jaccard3(sparse.csr_matrix(x),sparse.csr_matrix(y))
Out[82]: array([ 0.75])
```

(edit - explicitly use `sparse.kron`)

Share Edit Follow Flag

edited May 23, 2017 at 11:44



Community Bot

1 1

answered May 6, 2016 at 16:02



hpaulj

209k

13

213

320



Apologies for not being able to check this earlier, but I think your solution does not produce same result as `d.jaccard`. E.g, try with vectors `[1,1,0,0,1,0],[0,0,1,0,1,0]` – [Segmented](#) May 9, 2016 at 8:43



Does `jaccard_fast` get that case right? I tried to replicate that (except for the 1 0 switch). – [hpaulj](#) May 9, 2016 at 11:06



`jaccard_fast` does get this case right, I checked. Apologies, I have to look into this when I have access to computation devices again tomorrow. – [Segmented](#) May 9, 2016 at 12:10



My version(s) match; may be I'm misunderstanding your example. – [hpaulj](#) May 9, 2016 at 18:41



I get an `AttributeError: 'bool' object has no attribute 'sum'` at `dis = (mat[mask,:]!=aM).sum(1)` trying to run your tests. my scipy version is 0.17.0 and numpy underneath is 0.11.0 – [Segmented](#) May 10, 2016 at 9:51

|